

Agent Oxono

Douaa

1 Introduction

Dans le cadre de ce projet, nous avons développé un agent pour le jeu Oxono en utilisant des techniques de recherche adversariale. L'objectif est de construire un agent capable de jouer de manière compétitive face à différents types d'adversaires.

Nous avons choisi d'utiliser l'algorithme Alpha-Beta, qui permet d'explorer efficacement les coups possibles tout en réduisant le nombre de calculs nécessaires. Cependant, les performances de cet algorithme dépendent fortement de la fonction d'évaluation utilisée pour estimer la qualité des positions.

Au cours du développement, plusieurs versions de l'agent ont été testées. Nous avons progressivement amélioré la fonction d'évaluation, ajouté des heuristiques pour mieux gérer les situations d'attaque et de défense, ainsi qu'un mécanisme permettant de détecter certains coups critiques (comme une victoire immédiate ou un blocage nécessaire).

Enfin, nous avons réalisé plusieurs expériences afin de comparer les différentes versions de notre agent et analyser l'impact des améliorations apportées.

2 Conception de l'agent

2.1 Algorithme de base

Pour concevoir notre agent, nous avons utilisé l'algorithme Alpha-Beta, qui est une amélioration de l'algorithme Minimax. Cet algorithme est adapté aux jeux à deux joueurs comme Oxono, où chaque joueur cherche à maximiser ses chances de gagner tout en minimisant celles de l'adversaire.

Le principe est d'explorer les différents coups possibles sous forme d'un arbre de jeu. À chaque niveau, l'agent suppose que l'adversaire joue de manière optimale. L'algorithme Alpha-Beta permet d'éviter d'explorer certaines branches de cet arbre lorsque celles-ci ne peuvent pas influencer le résultat final, ce qui réduit fortement le temps de calcul.

Nous avons choisi cette approche car elle offre un bon compromis entre performance et simplicité d'implémentation. Elle permet également d'intégrer facilement une fonction d'évaluation pour estimer les positions lorsque la profondeur maximale de recherche est atteinte.

2.2 Fonction d'évaluation

La fonction d'évaluation est un élément central de notre agent, car elle permet d'estimer la qualité d'un état lorsque la profondeur maximale de recherche est atteinte.

Dans notre implémentation, nous avons utilisé une combinaison de plusieurs heuristiques :

- **Détection des états terminaux** : si l'état est terminal, nous retournons une valeur très élevée (ou très faible) en fonction du résultat afin de privilégier les victoires rapides et d'éviter les défaites.
- **Fenêtres de taille 4** : nous analysons différentes séquences de cases (alignements potentiels) pour évaluer les opportunités de victoire. Les configurations favorables à notre agent reçoivent un score positif, tandis que celles favorables à l'adversaire sont pénalisées.
- **Coups gagnants immédiats** : nous donnons une importance particulière aux situations où un joueur peut gagner au tour suivant. Ces configurations sont fortement pondérées afin de prioriser les actions critiques.

Cette combinaison permet à l'agent de prendre en compte à la fois les opportunités offensives et les menaces défensives.

2.3 Gestion du temps

Afin de respecter les contraintes de temps, nous avons mis en place une gestion adaptative de la profondeur de recherche.

Lorsque le temps restant est élevé, l'agent utilise une profondeur plus importante (par exemple 4). En revanche, lorsque le temps devient limité, la profondeur est réduite (par exemple 3) afin de garantir une réponse rapide.

Cette approche permet d'éviter les dépassements de temps tout en maintenant de bonnes performances.

2.4 Améliorations apportées

Plusieurs améliorations ont été intégrées afin d'optimiser les performances de l'agent :

- **Profondeur adaptative** : la profondeur de recherche est ajustée en fonction du temps restant. Lorsque le temps est suffisant, une profondeur plus grande est utilisée, sinon elle est réduite pour garantir une réponse rapide.
- **Élagage Alpha-Beta** : l'utilisation de l'algorithme Alpha-Beta permet de réduire considérablement le nombre de nœuds explorés, améliorant ainsi les performances.
- **Optimisation des coups critiques** : certaines situations, comme les victoires immédiates ou les blocages nécessaires, sont traitées en priorité afin d'éviter des erreurs coûteuses.

2.5 Version 1

La première version de notre agent utilisait une recherche Alpha-Beta avec une profondeur fixe et une fonction d'évaluation simple. Cette version permettait déjà de battre un agent aléatoire, mais restait limitée dans des situations plus complexes.

2.6 Version 2

Nous avons amélioré la fonction d'évaluation en ajoutant l'analyse des alignements (fenêtres de taille 4). Cette version a montré une meilleure capacité à anticiper les coups adverses.

2.7 Version 3

Nous avons introduit une gestion du temps permettant d'adapter la profondeur de recherche. Cela a permis d'obtenir de meilleures performances tout en respectant les contraintes de temps.

2.8 Version 4

Nous avons tenté d'ajouter des heuristiques supplémentaires pour détecter les coups critiques. Bien que certaines améliorations aient été observées, cette version a également introduit des comportements instables dans certains cas.

3 Approches non concluantes

Certaines améliorations testées n’ont pas donné les résultats escomptés :

- L’ajout de heuristiques trop complexes a parfois dégradé les performances, en introduisant du bruit dans l’évaluation.
- Une profondeur trop élevée a entraîné des dépassements de temps, rendant l’agent inutilisable dans certaines situations.
- Certaines tentatives d’optimisation ont introduit des bugs (par exemple des erreurs de calcul ou des comportements inattendus), ce qui a nécessité un retour à des versions plus stables.

Ces observations nous ont permis de mieux comprendre les compromis entre complexité, performance et stabilité.

4 Expériences

4.1 Configuration expérimentale

Nous avons évalué les performances de notre agent en le faisant jouer plusieurs parties contre différents adversaires, notamment un agent aléatoire.

Chaque configuration a été testée sur un nombre significatif de parties afin d’obtenir des résultats fiables.

4.2 Configuration expérimentale

Les performances de l’agent ont été évaluées en jouant plusieurs parties contre différents adversaires.

Chaque match a été répété plusieurs fois (environ 20 à 50 parties) afin d’obtenir des résultats fiables.

Les agents testés sont :

- Agent aléatoire
- Agent Alpha-Beta de base
- Agent MCTS

Adversaire	Win Rate	Nombre de parties
Random Agent	100%	30
Alpha-Beta Baseline	80%	30
MCTS Agent	55%	30

4.3 Évolution des performances

Version	Win Rate vs Random	Commentaire
V1	90%	Basique
V2	100%	Meilleure heuristique
V3	100%	Plus stable

5 Discussion

Les résultats montrent que l'amélioration de la fonction d'évaluation a un impact significatif sur les performances.

La gestion du temps permet également d'éviter les erreurs liées aux dépassements de temps, ce qui rend l'agent plus robuste.

Cependant, les performances contre l'agent MCTS restent limitées, ce qui montre que des améliorations sont encore possibles.

6 Conclusion

Dans ce projet, nous avons développé un agent pour le jeu Oxono basé sur l'algorithme Alpha-Beta.

Les différentes améliorations apportées, notamment au niveau de la fonction d'évaluation et de la gestion du temps, ont permis d'obtenir un agent performant capable de battre un agent aléatoire de manière systématique.

Cependant, certaines limites subsistent, notamment en termes de complexité et de stabilité. Des améliorations futures pourraient inclure l'utilisation d'algorithmes plus avancés, comme Monte Carlo Tree Search, ou l'optimisation de la fonction d'évaluation.

Ce projet nous a permis de mieux comprendre les enjeux liés à la recherche adversariale et à la conception d'agents intelligents.

7 to do

Comparaison avec MCTS 2. Baseline Alpha-Beta 3. Plus de détails dans les expériences